

Momentum → RMSProp → Adam

Three ways to turn a noisy gradient into a useful update direction.

THE QUESTION

How can an optimizer use past gradients to take a better step than the current gradient alone?

THE INTUITION

Keep running averages. An average of signed gradients remembers which direction persists; an average of squared gradients remembers how large each coordinate usually is. Adam divides the first by the root of the second.

KEY TAKEAWAY

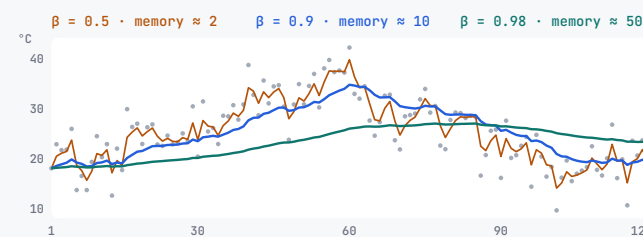
Momentum remembers direction and RMSProp remembers scale; Adam keeps both. Adaptive optimizers still need a tuned η .

minibatch $\hat{g}$ → m: EWMA of $\hat{g}$ → v: EWMA of $\hat{g}^2$ → bias-correct → $\hat{m}/(\sqrt{\hat{v}} + \epsilon)$ → $\theta \leftarrow \theta - \eta \cdot d$

1 Start with the optimizer contract

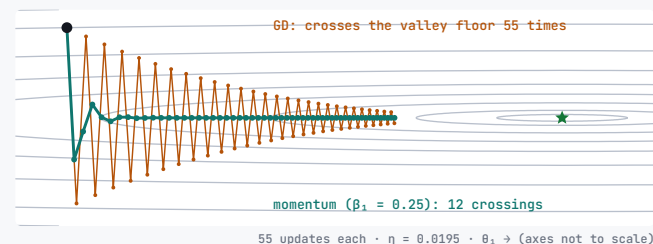
On minibatch $\mathcal{B}_t$ compute $\hat{g}_t = \nabla_{\theta} \mathcal{L}_{\mathcal{B}_t}(\theta_t)$. Every optimizer builds a direction d_t , then applies $\theta_{t+1} = \theta_t - \eta_t d_t$. The learning rate scales the move; the optimizer state changes its direction and its coordinate-wise scale.

2 EWMA: the shared memory primitive



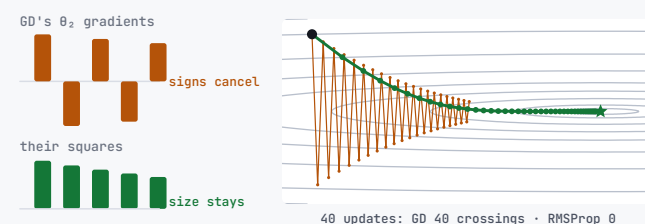
$m_t = \beta m_{t-1} + (1 - \beta)x_t$, $0 \leq \beta < 1$. Recent observations get weights $(1 - \beta), (1 - \beta)\beta, (1 - \beta)\beta^2, \dots$. **Effective memory:** roughly $1/(1 - \beta)$ observations. Worked: $m = 30, x = 36, \beta = 0.9$ gives $0.9 \cdot 30 + 0.1 \cdot 36 = 30.6$.

3 Momentum: is the direction consistent?



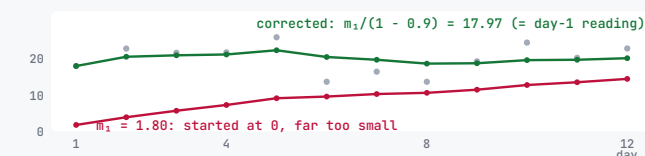
$m_{t+1} = \beta_1 m_t + (1 - \beta_1)\hat{g}_t$, $\theta_{t+1} = \theta_t - \eta m_{t+1}$. Components that keep their sign reinforce; oscillating ones cancel. First step: $m_1 = (-1.8, 63.75)$, loss 39.0 → 10.5 (GD: 35.4). Update 2 in θ_2 : history $-0.311 + \text{new} + 0.575 = +0.264$, against GD's $+1.575$; the bounce is damped while θ_1 keeps moving. With $\beta_1 = 0$ this is ordinary GD. Standardizing features may remove the ravine first. PyTorch `SGD(momentum=β)` keeps an unnormalized buffer $b \leftarrow \beta b + \hat{g}$, so its lr equals $\eta(1 - \beta_1)$ here ($0.0195 \times 0.75 = 0.014625$).

4 RMSProp: large relative to usual?



$v_{t+1} = \beta_2 v_t + (1 - \beta_2)\hat{g}_t^2$, $\theta_{t+1} = \theta_t - \eta \hat{g}_t / (\sqrt{v_{t+1}} + \epsilon)$. Squaring stops sign cancellation; dividing by the recent root-mean-square gives each coordinate its own ruler. First update here: $\sqrt{v} = (0.76, 26.9)$, so $\hat{g}/\sqrt{v} = (-3.16, 3.16)$ and both coordinates move by the same 0.253. v is a raw second moment, not a centred variance; all operations are elementwise.

5 Adam: direction and scale



$m_{t+1} = \beta_1 m_t + (1 - \beta_1)\hat{g}_t$, $v_{t+1} = \beta_2 v_t + (1 - \beta_2)\hat{g}_t^2$
 $\hat{m}_{t+1} = m_{t+1}/(1 - \beta_1^{t+1})$, $\hat{v}_{t+1} = v_{t+1}/(1 - \beta_2^{t+1})$
 $\theta_{t+1} = \theta_t - \eta \hat{m}_{t+1}/(\sqrt{\hat{v}_{t+1}} + \epsilon)$.

Both memories start at zero, so early values are too small (above: $\beta = 0.9$ on the temperatures). Dividing by $1 - \beta^t$ fixes this, mostly in the first updates.

6 What each method stores

method	state	question it answers
GD	none	what is the gradient now?
Momentum	m	which direction persists?
RMSProp	v	which coordinates run large?
Adam	m, v	which direction, at what scale?

7 Pause and think

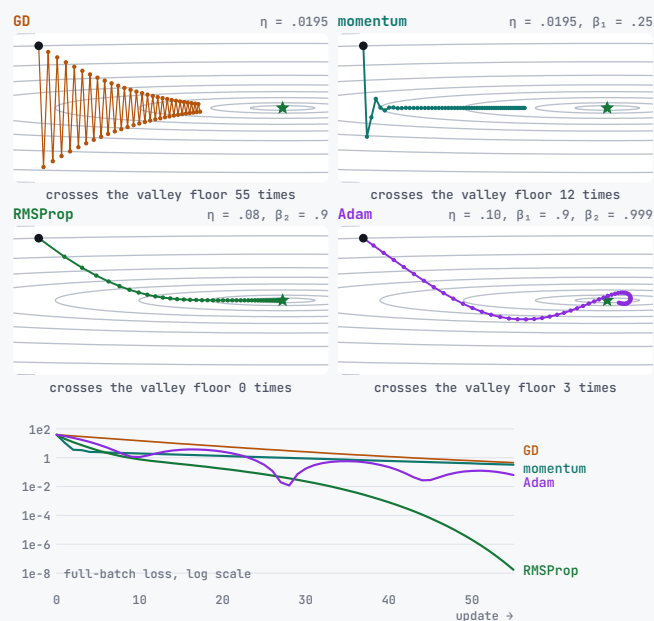
Why square before averaging? GD's θ_2 gradients 85, -80.75 , 76.71, ... nearly cancel; their squares 7225, 6521, ... keep the size.

Why can RMSProp use $\eta = 0.08$ when GD needs 0.0195? Dividing by $\sqrt{v}$ puts both coordinates on one scale; the steep θ_2 no longer caps η .

Use adaptive optimizers deliberately

State initialization, step order, diagnostics, and fair comparisons.

8 Four optimizers, one ravine



Same start, 55 full-batch updates on $\mathcal{L} = \frac{1}{2}(\theta_1 - 1)^2 + 50(\theta_2 + 0.5)^2$, with the slides' method-specific rates. It shows how each method moves and does not rank them: RMSProp looks ideal only because this ravine is aligned with the axes.

9 One correct Adam step

- 1 compute the current minibatch gradient
- 2 update first-moment state m
- 3 update second-moment state v
- 4 bias-correct using the update count $t + 1$
- 5 divide $\hat{m}$ by $\sqrt{\hat{v}} + \epsilon$
- 6 apply the learning-rate-scaled move

Initialize $m = v = 0$ once, not every minibatch.
 Worked on the ravine ($\eta = 0.1$): $m_1 = (-0.24, 8.5)$ and $\hat{m}_1 = (-2.4, 85) = \hat{g}_0$; $\sqrt{\hat{v}_1} = (2.4, 85)$; step = $(+0.1, -0.1)$. The first Adam step is $\pm\eta$ per coordinate.

10 Defaults are starting points

knob	typical	effect
η	10^{-3}	global step scale; tune first
β_1	0.9	higher = longer direction memory
β_2	0.999	higher = longer scale memory
ϵ	10^{-8}	numerical floor; watch low precision

$\eta = 10^{-3}$ is the usual Adam starting point. Its useful range depends on architecture, normalization, batch size, loss scale and optimizer convention. The toy runs use larger rates.

11 PyTorch training order

```
opt = optim.Adam(model.parameters(), lr=1e-3)
for x, y in loader:
    opt.zero_grad(set_to_none=True)
    loss = criterion(model(x), y)
    loss.backward()
    opt.step()
```

Gradients accumulate unless cleared. The optimizer owns its per-parameter state (m, v , step count); the model owns parameters and gradients.

AdamW applies weight decay as a separate shrink instead of mixing it into the adaptively scaled gradient. Treat the decay coefficient as a tuned regularization knob.

12 Diagnose the symptom

Explodes / NaN: lower η ; check data, loss scale, logits.
Ravine zigzag: standardize; try momentum; lower η .
One coordinate dominates: check feature scale; try RMSProp or Adam.
No early progress: check gradients and state; raise η carefully.
Good train, poor validation: generalization, not only the optimizer.

13 Compare optimizers fairly

- ✓ same split, shuffling, init and model
- ✓ same update budget and logging
- ✓ tune each optimizer's η separately
- ✓ report validation, not only train loss
- ✓ several seeds when differences are small
- ✓ log gradient norms, update norms, state

14 Invariants worth asserting

Shapes: θ, g, m, v match elementwise. **Finiteness:** loss, gradients and states contain no NaN/Inf. **Clock:** bias correction counts parameter updates. **State:** checkpoint the optimizer state when resuming.

Optimizers change the path, not the objective. Inspect training dynamics and validation behaviour.

Numbers from the Lecture 7 slides' examples (the seed-12 temperature signal; the ravine from Lecture 5), recomputed in `briefs/L07-data.py`. Slides and lesson: nipunbatra.github.io/dl-2026.